



CloseApps

Open

Fecha aplicativos abertos via CLI · Windows

Documentação do Projeto

Utilitário de linha de comando para fechar aplicativos abertos no Windows

Versão Plataforma

Autor

Data

v1.0.0 Windows · .NET 10 Maycon Wisley 01 de julho de 2026

LICENÇA MIT

Sumário

1. Visão Geral

2. Instalação

2.1 Requisitos

2.2 Publicação e instalação no PATH

3. Uso via Linha de Comando

3.1 Sintaxe

3.2 Opções disponíveis

3.3 Exemplos

4. Modo Interativo

5. Comportamento de Fechamento

5.1 Fechamento gentil vs. forçado

5.2 Critérios de exclusão de processos

5.3 Códigos de saída

6. Arquitetura e Estrutura do Projeto

7. Build, Testes e Integração Contínua

8. Licença

1. Visão Geral

CloseAppsOpen é um utilitário de linha de comando para Windows que fecha aplicativos com janela visível de forma rápida e controlada. Foi criado para agilizar rotinas do dia a dia — liberar memória antes de uma tarefa pesada, encerrar tudo antes de desligar o computador, ou fechar um conjunto específico de programas por nome — sem precisar clicar janela por janela.

O programa oferece dois modos de uso: um **modo interativo**, com menu no terminal, e um **modo direto**, controlado por argumentos de linha de comando, pensado para scripts e atalhos no `PATH` do sistema.

Destaque técnico: o motor de fechamento (`ProcessManager`) suporta dois modos de encerramento — gentil (`CloseMainWindow`) e forçado (`Kill`) — descritos em detalhe na Seção 5.

2. Instalação

2.1 Requisitos

- Windows 10/11
- [.NET 10](#) — dispensável se o executável for publicado como `self-contained`

2.2 Publicação e instalação no PATH

Publique o executável single-file e adicione-o ao `PATH` do Windows:

```
dotnet publish CloseAppsOpen\CloseAppsOpen.csproj -c Release -r win-x64 `
  --self-contained true -p:PublishSingleFile=true -o publish
```

Mova o arquivo `publish\CloseAppsOpen.exe` para uma pasta presente no `PATH` (por exemplo, `C:\Tools`) e utilize a partir de qualquer terminal:

```
closeappsopen --help
```

3. Uso via Linha de Comando

3.1 Sintaxe

```
closeappsopen [opções]
```

3.2 Opções disponíveis

Flag	Descrição
<code>-a, --all</code>	Fecha todos os aplicativos abertos
<code>-s, --shutdown</code>	Fecha tudo e desliga o PC
<code>-k, --kill <nome></code>	Fecha processos cujo nome ou título contenha <code><nome></code> (pode repetir a flag)
<code>-l, --list</code>	Lista os aplicativos abertos e encerra
<code>-e, --exclude <nome></code>	Exclui um processo pelo nome (pode repetir a flag)
<code>-f, --force</code>	Mata os processos diretamente (<code>kill</code>), sem confirmação e sem tentar o fechamento gentil
<code>-t, --timeout <ms></code>	Tempo de espera antes de forçar o encerramento (padrão: <code>2000</code> ms)
<code>-v, --version</code>	Exibe a versão instalada
<code>-h, --help</code>	Exibe a ajuda de uso

3.3 Exemplos

```
closeappsopen           # Abre o menu interativo
closeappsopen --list    # Lista os apps abertos
closeappsopen --all     # Fecha tudo, com confirmação
closeappsopen --all --force # Mata tudo na hora, sem perguntar
closeappsopen --kill chrome # Fecha processos com 'chrome' no nome
closeappsopen --all -e explorer # Fecha tudo, exceto o Explorer
closeappsopen -a -e chrome -e slack # Fecha tudo, exceto Chrome e Slack
closeappsopen --timeout 5000 --all # Aguarda 5s antes de forçar o encerramento
closeappsopen --shutdown # Fecha tudo e desliga o PC (com confirmação)
closeappsopen --shutdown --force # Fecha tudo e desliga, sem perguntar
```

4. Modo Interativo

Executado sem argumentos, o programa abre um menu no terminal com a lista de aplicativos que possuem janela visível no momento:

Tecla	Ação
A	Fecha todos os aplicativos listados (pede confirmação)
S	Permite selecionar quais fechar — por número, nome ou digitando <code>todos</code>
D	Fecha tudo e desliga o computador
R	Atualiza a lista de aplicativos
Q	Sai do programa

Todas as ações do menu interativo respeitam as flags globais informadas na chamada do programa, como `--timeout`, `--exclude` e `--force`.

5. Comportamento de Fechamento

5.1 Fechamento gentil vs. forçado

O encerramento de cada processo pode ocorrer de duas formas, controladas pela flag `--force` :

Modo	Mecanismo	Diálogo de confirmação do app	Confirmação do CLI
Padrão	<code>CloseMainWindow()</code> e, se o processo não sair dentro do <code>--timeout</code> , <code>Kill()</code>	Pode aparecer (ex.: "Salvar alterações?")	Pede confirmação
<code>--force</code>	<code>Kill()</code> imediato	Nunca aparece	Não pede

No modo padrão, `CloseMainWindow()` envia a mensagem `WM_CLOSE` — o equivalente a clicar no **X** da janela. Isso dá ao aplicativo a chance de perguntar sobre alterações não salvas (como ocorre em apps do pacote Office). Se o processo não encerrar dentro do tempo configurado em `--timeout` (padrão de 2000 ms), o programa força o encerramento com `Kill()` .

Atenção com `--force`: nesse modo o processo é encerrado via `Kill()` diretamente, sem passar por `CloseMainWindow()` . Isso significa que diálogos de confirmação do próprio aplicativo nunca são exibidos e qualquer trabalho não salvo é descartado sem aviso. Use com cautela em aplicativos com documentos abertos.

5.2 Critérios de exclusão de processos

A listagem de aplicativos (`GetVisible`) aplica os seguintes filtros antes de exibir ou fechar qualquer processo:

- Somente processos com `MainWindowHandle` válido e título de janela não vazio;
- O próprio processo do `CloseAppsOpen` é sempre excluído;
- Toda a cadeia de processos ancestrais (shell, `conhost` / `OpenConsole` , Windows Terminal) que hospeda o programa é excluída, para não fechar o terminal em uso;
- Sob o Windows Terminal, como a comunicação ocorre via pseudoconsole (ConPTY), o processo `WindowsTerminal.exe` não aparece na árvore de ancestrais — por isso o programa também compara o título da janela com o título do próprio console para excluí-lo com segurança;
- Nomes informados via `--exclude` são removidos da lista.

5.3 Códigos de saída

Código	Significado
0	Sucesso — todos os processos-alvo foram fechados (ou nenhuma ação era necessária)
1	Falha — pelo menos um processo não pôde ser fechado, ou nenhum processo correspondeu ao filtro em <code>--kill</code>

O código de saída permite o uso do `CloseAppsOpen` em scripts de automação, verificando o resultado da execução.

6. Arquitetura e Estrutura do Projeto

O projeto é organizado em uma solução .NET com dois projetos: a aplicação principal e a suíte de testes unitários.

CloseAppsOpen/

CloseAppsOpen/

Program.cs

CliArgs.cs

ProcessManager.cs

PowerManager.cs

ConsoleUI.cs

InteractiveMode.cs

app.ico

CloseAppsOpen.Tests/

assets/

tools/

Ponto de entrada e roteamento de flags

Parsing dos argumentos de linha de comando

Listagem e encerramento de processos (WinAPI)

Desligamento do PC (shutdown /s)

Toda a saída/entrada do console

Menu interativo

Ícone do executável

Testes unitários (xUnit)

Logo do projeto

Script gerador do app.ico

Componentes principais

Módulo	Responsabilidade
CliArgs	Interpreta os argumentos recebidos e expõe um objeto tipado com as opções ativas
ProcessManager	Usa a API do Windows (Toolhelp32Snapshot) para mapear a árvore de processos, listar janelas visíveis e executar o fechamento gentil ou forçado
PowerManager	Invoca o comando shutdown do Windows para desligar a máquina
ConsoleUI	Centraliza toda a formatação de saída no terminal (menus, tabelas, mensagens de ajuda)
InteractiveMode	Orquestra o loop do menu interativo, delegando o fechamento ao ProcessManager
Program	Ponto de entrada — decide entre modo direto (flags) e modo interativo

7. Build, Testes e Integração Contínua

Comandos locais

```
dotnet build
dotnet test
dotnet run -- --help
```

Pipeline de CI/CD

O workflow `.github/workflows/ci.yml` é executado automaticamente a cada `push` e `pull request` na branch `master`, e também na publicação de uma nova release:

Etapa	Descrição
Restore	Restaura as dependências da solução (<code>CloseAppsOpen.slnx</code>)
Build	Compila a solução em modo <code>Release</code>
Test	Executa a suíte de testes unitários com xUnit
Publish	Gera o <code>CloseAppsOpen.exe</code> self-contained (win-x64) e anexa o binário à release do GitHub — executado apenas quando uma release é publicada

Nota: o job de build roda em `windows-latest`, já que a aplicação depende de APIs nativas do Windows (Toolhelp32Snapshot, shutdown).

8. Licença

Este projeto é distribuído sob a **Licença MIT**, permitindo uso, cópia, modificação e distribuição livres, inclusive para fins comerciais, desde que o aviso de copyright original seja mantido.

MIT License

Copyright (c) 2026 Maycon Wisley

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.